

Задание 3. Разведочный анализ

Сайт: [Сервер дистанционного обучения Лаборатории ОИТ](#)

Курс: 2 семестр Анализ данных в Python

Книга: Задание 3. Разведочный анализ

Напечатано.: Виталий Евгеньевич Ким

Дата: Thursday, 16 July 2026, 05:38

Оглавление

1. Введение

2. 1. Данные опроса

3. 2. Добавление названия видов в набор данных

4. 3. Добавьте координаты из локаций участка

1. Введение

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
plt.style.use('seaborn-whitegrid')
```

Данные наблюдений за видами (когда и где наблюдается данный вид) типичны для исследований биоразнообразия. Крупные международные инициативы поддерживают сбор этих данных волонтерами, например, [iNaturalist](#). Благодаря таким инициативам, как [GBIF](#), многие из этих данных также находятся в открытом доступе.

Мы решили поделиться данными полевой кампании, но набор данных все еще требует некоторой очистки и стандартизации. Например, координаты могут называться `x / y`, `decimalLatitude / decimalLongitude`, `lat / long` ... К счастью, вы знаете о международном **стандарте открытых данных** для описания данных о событиях/наблюдениях, т.е. [Darwin Core \(DwC\)](#). Вместо того, чтобы изобретать собственную модель данных, вы решаете соответствовать этому международному стандарту. Последнее улучшит коммуникацию, а также сделает ваши данные совместимыми с GBIF.

Короче говоря, DwC описывает плоскую таблицу (CSV) с согласованным соглашением об именах для имен заголовков и соглашениями о том, как должны быть представлены определенные типы данных (для справки, подробное описание дано [здесь](#)). В этом руководстве мы сосредоточимся на нескольких существующих терминах, чтобы изучить некоторые элементы очистки данных:

`eventDate` : формат дат ISO 6801

`ScientificName` : общепринятое научное название вида.

`decimalLatitude / decimalLongitude` : координаты вхождения в формате WGS84

`sex`: либо мужской, либо женский, чтобы охарактеризовать пол события

`instanceID` : идентификатор в наборе данных для идентификации отдельных записей.

`datasetName` : статическая строка, определяющая источник данных

Кроме того, дополнительная информация о таксономии будет добавлена с использованием внешнего API-сервиса.

Набор данных для работы:

Для этого набора данных данные разделены на следующие основные файлы данных:

[Surveys.csv](#) данные опросов на отдельных участках

[species.csv](#) обзорный список кратких названий видов

[plot_location.xlsx](#) обзор координат отдельных локаций

Данные получены из [исследования](#) экосистемы пустыни Чиуауа недалеко от Портала, штат Аризона.



2. 1. Данные опроса

Чтение данных отдельных опросов:

```
Survey_data = pd.read_csv ( "data/surveys.csv" )
```

```
Survey_data.head()
```

Упражнение 1.

Сколько уникальных записей содержит набор данных?

Добавление информации об источнике данных в виде статического столбца

Для удобства, когда этот набор данных будет объединяться с другими наборами данных, мы сначала добавляем столбец статических значений, определяя datasetName этих конкретных данных:

```
datasetname = "Ecological Archives E090-118-D1."
```

УПРАЖНЕНИЕ 2

Добавьте новый столбец datasetName в набор данных опроса с datasetname в качестве значения для всех записей (статическое значение для всего набора данных).

Очистка столбца sex_char в Dwc,

УПРАЖНЕНИЕ 3

Получите список уникальных значений для столбца sex_char.

Таким образом, по-видимому, в этом столбце содержится больше информации, тогда как, согласно информации [метаданных](#), информация о поле должна быть либо M (мужской), либо F (женский). Мы создадим столбец с именем пол и преобразуем символы в соответствующий пол, принимая во внимание следующее сопоставление значений (подробнее см. [метаданные](#)):

M -> мужчина

Ж -> женщина

P -> мужчина

П -> женщина

Z -> nan

В то же время мы сохраним исходную информацию sex_char в отдельном столбце с именем verbatimSex в качестве ссылки на случай, если нам понадобятся исходные данные позже.

Подводя итог, мы должны:

переименуйте столбец sex_char в verbatimSex

создайте новый столбец с названием секс

сопоставьте исходные значения sex_char со значениями male и female в соответствии с приведенным выше сопоставлением

Во-первых, давайте преобразуем имя заголовка столбца sex_char в verbatimSex с помощью функции переименования :

```
survey_data = survey_data.rename(columns={'sex_char': 'verbatimSex'})
```

УПРАЖНЕНИЕ 4

Выразите отображение значений (например, M -> male) в объект словаря Python с именем переменной sex_dict. Значения Z соответствуют Not a Number, который можно определить как np.nan.

Используйте словарь sex_dict, чтобы заменить значения в столбце verbatimSex новыми значениями и сохранить сопоставленные значения в новом столбце «пол» кадра данных.

Проверка текущей частоты значений результирующего столбца пола (в результате должны получиться значения male , female и nan):

```
survey_data["sex"].unique()
```

Чтобы проверить, какова частота появления мужчин и женщин в категориях, гистограмма является возможным представлением:

УПРАЖНЕНИЕ 5

Создайте горизонтальную гистограмму, сравнивая количество записей о мужчинах, женщинах и неизвестных (NaN) в наборе данных.

Разрешение поля двойной записи путем разделения

При проверке видовой уникальной информации:

```
survey_data["species"].unique()
```

```
survey_data.head(10)
```

По-видимому, существует двойная запись: «DM и SH», которая в основном определяет две записи и должна быть разделена на две отдельные записи (т. е. строки). Следовательно, мы должны иметь возможность создать дополнительную строку на основе этого разделения. Для этого Pandas предоставляет специальную функцию, начиная с версии 0.25, которая называется `explode`. Начиная с небольшого примера подмножества:

```
example = survey_data.loc[7:10, "species"]
```

Используя метод разделения строк, мы можем разделить строку, используя заданный символ, в данном случае слово и :

```
example.str.split("and")
```

Метод разнесения создаст строку для каждого элемента в списке:

```
example_split = example.str.split("and").explode()
example_split
```

Следовательно, DM и SH теперь зачисляются в отдельные строки. Остальные строки остаются без изменений. Единственная оставшаяся проблема - это пробелы вокруг символов:

```
example_split.iloc[ 1 ], example_split.iloc[ 2 ]
```

Которую мы можем решить снова с помощью строкового метода `strip`, удалив пробелы до и после символов:

```
example_split.iloc[1], example_split.iloc[2]
```

Чтобы сделать это многократно, давайте создадим специальную функцию для объединения этих шагов, называемую `solve_double_field_entry` :

```
def solve_double_field_entry(df, keyword="and", column="verbatimEventDate"):
    """Разделить по ключевому слову в столбце для перечисления и создать дополнительную запись
```

Параметры

df: pd.DataFrame

DataFrame с двойной записью поля в одном или нескольких значениях

keyword: str

слово/символ для разделения двойные записи в

column: имя столбца str

для разделения записей

"""

```
df = df.copy() # copy the input DataFrame to avoid editing the original
df[column] = df[column].str.split(keyword)
df = df.explode(column)
df[column] = df[column].str.strip() # remove white space around the words
return df
```

Функция принимает DataFrame в качестве входных данных, разбивает запись на отдельные строки и возвращает обновленный DataFrame . Мы можем использовать эту функцию, чтобы получить обновление DataFrame с дополнительной строкой (наблюдением), добавленной путем разделения определенного поля. Давайте применим эту новую функцию.

УПРАЖНЕНИЕ 6

Используйте функцию `solve_double_field_entry`, чтобы обновить данные обзора, отделив двойные записи. Сохраните результат как переменную `Survey_data_decoupled`.

```
survey_data_decoupled["species"].unique()
```

```
survey_data_decoupled.head(11)
```

Создать новый идентификатор вхождения

Record_id больше не является уникальным идентификатором для каждого наблюдения после отделения этого набора данных. Мы создадим специальный идентификатор нового набора данных, добавив столбец с именем instanceID , который принимает новый счетчик в качестве идентификатора. В качестве простого и понятного подхода мы будем использовать новый счетчик для всего набора данных, начиная с 1:

```
np.arange( 1 , len (survey_data_decoupled) + 1 , 1 )
```

Чтобы создать новый столбец с идентификатором вхождения заголовка со значениями 1 -> 35550 в качестве значений поля:

```
Survey_data_decoupled[ "occurrenceID" ] = np.arange( 1 , len (survey_data_decoupled) + 1 , 1 )
```

Чтобы устранить путаницу, связанную с наличием поля record_id и вхождения ID , мы удалим термин record_id :

```
urvey_data_decoupled = survey_data_decoupled.drop(columns="record_id")
```

Следовательно, столбцы могут быть удалены из DataFrame .

```
Survey_data_decoupled.head( 10 )
```

Преобразование значений даты

В полученном наборе данных опроса столбцы месяц , день и год содержат информацию о дате, т.е. eventDate в терминах DarwinCore. Нам нужны эти данные в формате ISO ГГГГ-ММ-ДД . Удобная функция Pandas — это использование to_datetime , которая предоставляет несколько вариантов интерпретации дат. Одним из вариантов является автоматическая интерпретация некоторых «типичных» столбцов, таких как year , month и day , при передаче DataFrame .

```
# pd.to_datetime(survey_data_decoupled[["year", "month", "day"]]) # раскомментируйте строку и проверьте это утверждение
```

Это не работает, не все даты можно интерпретировать... Мы должны получить больше информации о причине ошибок. При использовании параметра coerce создатели проблемы будут помечены как отсутствующее значение NaN . Мы можем подсчитать количество дат, которые не могут быть интерпретированы:

```
sum(pd.to_datetime(survey_data_decoupled[["year", "month", "day"]], errors='coerce').isna())
```

УПРАЖНЕНИЕ 7

Сделайте выборку Survey_data_decoupled, содержащую те записи, которые не могут быть правильно интерпретированы как значения даты, и сохраните полученный DataFrame как новую переменную Trouble_makers.

Проверяем некоторые характеристики возмутителей спокойствия:

```
trouble_makers.head()
```

```
trouble_makers["day"].unique()
```

```
trouble_makers["month"].unique()
```

```
trouble_makers["year"].unique()
```

Проблема заключается в наличии дня 31 в апреле и сентябре 2000 года. В этот момент нам пришлось бы перепроверить исходные данные, чтобы узнать, как можно решить проблему. По-видимому, для этого конкретного случая в 2000 году возникла проблема с вводом данных, из-за чего 31 день в течение этого периода на самом деле должен был быть 30 . Было бы оптимально исправить это в исходном наборе данных, но для упражнения мы исправим это здесь.

УПРАЖНЕНИЕ 8

Присвойте во фрейме данных Survey_data_decoupled всем значениям дня нарушителей спокойствия значение 30 вместо 31.

Теперь мы снова выполняем синтаксический анализ, чтобы создать правильное поле eventDate , содержащее даты:

```
survey_data_decoupled["eventDate"] = \
pd.to_datetime(survey_data_decoupled[["year", "month", "day"]])
```

УПРАЖНЕНИЕ 9

Проверьте количество наблюдений за каждый год. Создайте горизонтальную гистограмму с количеством строк/наблюдений за каждый год.

```
survey_data_decoupled.head()
```

В настоящее время даты хранятся в формате даты, специфичном для Python:

```
Survey_data_decoupled[ "eventDate" ].dtype
```

Это здорово, потому что позволяет использовать множество функций с помощью аксессора .dt :

Survey_data_decoupled.eventDate.dt # добавьте точку (.) и нажмите клавишу TAB, чтобы изучить доступные варианты даты

УПРАЖНЕНИЕ 10

Создайте горизонтальную гистограмму с количеством записей для каждого года (см. выше), но без использования столбца года, используя непосредственно столбец eventDate.

На самом деле нам больше не нужны столбцы день , месяц , год , но не стесняйтесь использовать то, что вам больше подходит.

УПРАЖНЕНИЕ 11

Создайте гистограмму с количеством записей для каждого дня недели (dayofweek)

При сохранении информации в файл (например, CSV -файл) этот тип данных будет автоматически преобразован в строковое представление. Тем не менее, мы также можем решить явно предоставить строковый формат, в котором хранятся даты (теряя функциональные возможности типа даты), чтобы иметь полный контроль над форматированием этих дат:

```
survey_data_decoupled["eventDate"] = survey_data_decoupled["eventDate"].dt.strftime("%Y-%m-%d")
```

```
survey_data_decoupled["eventDate"].head()
```

Удалим столбцы день/год/месяц.

```
survey_data_decoupled = survey_data_decoupled.drop(columns=["day", "month", "year"])
```


3. 2. Добавление названия видов в набор данных

Вид столбца предоставляет только краткий идентификатор в обзоре опроса. Информация об имени хранится в отдельном файле `species.csv`. Мы хотим, чтобы наш набор данных включал эту информацию, считывал данные и добавлял их в наш набор данных опроса:

УПРАЖНЕНИЕ 12

Прочитайте файл «`species.csv`» и сохраните полученный набор данных как переменную `species_data`.

```
species_data.head()
```

Исправить неправильное название аббревиатуры

При просмотре метаданных вы видите, что в файле данных для описания *Neotoma albigula* используется аббревиатура NE, тогда как в [описании метаданных](#) используется аббревиатура NA.

УПРАЖНЕНИЕ 13

Преобразуйте значение «NE» в «NA», используя логическое индексирование/фильтрацию для столбца `species_id`.

Объединение съемок и видов

Теперь, когда мы подготовили две серии, мы можем объединить данные, снова используя операцию `pd.merge`.

Мы хотим добавить данные о видах к данным обследования, чтобы увидеть полные названия видов в сводной таблице данных.

УПРАЖНЕНИЕ 14

Объедините фреймы данных `Survey_data_plots` и фрейм данных. видов_данных путем добавления соответствующей информации о видах (название, класс, царство и т. д.) к отдельным наблюдениям. Назначьте выходные данные новой переменной `Survey_data_species`.

```
survey_data_species = pd.merge(survey_data_decoupled, species_data, how="left", left_on="species", right_on="species_id")
```

```
len(survey_data_species) # проверка длины после операции соединения
```

Соединение в порядке, но у нас остались некоторые избыточные столбцы и неправильное название:

```
Survey_data_species.head()
```

видов_x и видов_ид нам больше не нужны, так как с этого момента мы будем использовать научные имена:

```
Survey_data_species = Survey_data_species.drop(["species_x", "species_id"], axis = 1)
```

Столбец видов_y может быть просто назван видами:

```
Survey_data_species = Survey_data_species.rename (columns = {"species_y": "species"})
```

```
Survey_data_species.head()
```

```
len(survey_data_species)
```

4. 3. Добавьте координаты из локаций участка

Загрузка данных координат

Отдельные участки идентифицируются только идентификационным номером участка. Для предоставления достаточной информации внешним пользователям необходимо добавить дополнительную информацию о координатах. Координаты отдельных участков сохраняются в другом файле: `plot_location.xlsx`. Мы будем использовать эту информацию для дальнейшего обогащения нашего набора данных и добавления основных терминов Дарвина `decimalLongitude` и `decimalLatitude`.

УПРАЖНЕНИЕ 15

Прочитайте файл excel '`plot_location.xlsx`' и сохраните данные как переменную `plot_data` с 3 столбцами: `plot`, `xutm`, `yutm`.

```
plot_data.head()
```

Преобразование в другую систему отсчета координат

Эти координаты указаны в метрах, а точнее в системе координат [UTM 12 N](#). Однако согласованным представлением координат для Darwin Core является [Всемирная геодезическая система 1984 года \(WGS84\)](#).

Поскольку это не курс ГИС, мы сократим обсуждение различных систем проекций, но приведем пример того, как такое преобразование из UTM12N в WGS84 может быть выполнено с помощью инструментария проекций `pyproj` и с использованием существующих кодов EPSG (реестр первоначально созданной ассоциацией производителей нефти и газа).

Во-первых, мы определяем две проекционные системы, используя соответствующие им коды EPSG:

```
from pyproj import Transformer
```

```
transformer = Transformer.from_crs("EPSG:32612", "epsg:4326")
```

может быть выполнено с помощью функции преобразования набора инструментов проецирования, предоставляющей системы координат и набор координат `x`, `y`. Например, для одной координаты это можно применить следующим образом:

```
transformer.transform(681222.131658, 3.535262e+06)
```

Такое преобразование — функция, не поддерживаемая самим Pandas (она есть на <https://geopandas.org/>). В такой ситуации мы хотим применить пользовательскую функцию к каждой строке `DataFrame`. Вместо того, чтобы писать цикл `for` для каждой из координат в списке, мы можем `.apply()` эту функцию с Pandas.

УПРАЖНЕНИЕ 16

Примените преобразование функции `pyproj` к `plot_data`, используя столбцы `xutm` и `yutm`, и сохраните результат в двух новых столбцах, называемых `decimalLongitude` и `decimalLatitude`:

Создайте функцию `transform_utm_to_wgs`, которая берет строку `DataFrame` и возвращает серию из двух элементов с долготой и широтой.

Протестируйте эту функцию в первой строке `plot_data`.

Теперь примените эту функцию ко всем строкам (используйте правильный параметр оси)

Назначьте результат предыдущего шага столбцам `decimalLongitude` и `decimalLatitude`.

```
plot_data.head()
```

Вышеупомянутая функция `transform_utm_to_wgs`, которую вы создали, является очень специфической функцией, которая знает структуру `DataFrame`, к которой вы будете ее применять (она предполагает имена столбцов «`xutm`» и «`yutm`»). Мы также могли бы создать более общую функцию, которая просто принимает координаты `X` и `Y` и возвращает ряд преобразованных координат (`transform_utm_to_wgs2(X, Y)`).

Альтернатива применению такой пользовательской функции к `plot_data DataFrame` — это использование конструкции `lambda`, которая позволяет указать функцию в одной строке в качестве аргумента:

```
трансформатор = Transformer.from_crs("EPSG:32612", "epsg:4326")
```

```
plot_data.apply(лямбда-строка: трансформатор.transform(строка['xutm'], строка['yutm']), ось=1)
```

Присоедините информацию о координатах к набору данных съемки

Мы можем расширить наш набор геодезических данных с помощью этой информации о координатах. Объединение двух наборов данных на основе общего идентификатора полностью аналогично использованию операций JOIN в базах данных. В Pandas эту функциональность предоставляет [pd.merge](#).

На практике мы должны добавить столбцы decimalLongitude / decimalLatitude к текущему набору данных Survey_data_decoupled , используя идентификационный номер участка в качестве ключа для соединения.

УПРАЖНЕНИЕ 17

Извлеките только столбцы для присоединения к нашему набору данных опроса: идентификаторы участков, decimalLatitude и decimalLongitude в новую переменную с именем plot_data_selection.

УПРАЖНЕНИЕ 18

Объедините DataFrame plot_data_selection и DataFrame Survey_data_decoupled путем добавления соответствующей информации о координатах к отдельным наблюдениям с помощью функции pd.merge() . Назначьте выходные данные новой переменной Survey_data_plots.

```
Survey_data_plots.head()
```

Местоположение графика должно быть сохранено с именем переменной verbatimLocality , указывающим идентификатор как целочисленное значение графика:

```
Survey_data_plots = Survey_data_plots.rename (столбцы = { 'plot' : 'verbatimLocality' })
```

Теперь давайте сохраним наши чистые данные в CSV - файл, чтобы мы могли продолжить анализ данных в следующем блокноте:

```
Survey_data_plots.to_csv ( "interim_survey_data_species.csv" , index = False )
```